

**SUMMER INTERNSHIP REPORT
OF
AgenticOS: HARNESS**

Submitted
To

School of Computer Science and Engineering
IILM University, Greater Noida

In partial fulfilment of the
requirement of degree of B.Tech
CSE (AI/ML)



By

MUDIT AGRAWAL

CS-2341718

2026-27

7TH Semester | 4CSE1

Organization: Carnot Research Pvt. Ltd

CANDIDATE'S DECLARATION

I, Mudit Agrawal, do hereby declare that the internship report titled “**AgenticOS : HARNESS**” has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence & Machine Learning). All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own work and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Signature: _____



ACKNOWLEDGEMENT

I use this opportunity to express my gratitude to everyone who supported me throughout the Internship Program at **Carnot Research Pvt. Ltd.** I am thankful for their guidance valuable constructive criticism and friendly advice throughout the Internship Program. I am sincerely grateful to **Dr. Amit Oberoi** and the **Development Team** for their mentorship and for the opportunity to work on a production-grade AI project. I would also like to thank Ms. Kanica Sachdev and Mr. Pranav Kanire at Carnot Research Pvt. Ltd. For their coordination and support throughout the Internship Program.

I would also like to thank the faculty of the **School of Computer Science and Engineering at IILM University in Greater Noida** for their guidance and for giving me the chance to do this industry internship, as part of the curriculum.

I express my gratitude to my parents for their blessings and constant support.

TABLE OF CONTENTS

Chapter No.	Chapter Name	Page No.
	Cover Page	1
1.	Candidate's Declaration	2
2.	Acknowledgement	3
3.	Internship Completion Certificate	5
4.	Project Description	6
	4.1 Introduction	6
	4.2 Organization Profile	6
	4.3 Problem Statement	7
	4.4 Project Objectives	8
	4.5 Scope of the Project	9
	4.6 Technologies and Tools Used	9
	4.7 System Architecture	10
	4.8 Project Flow	10
	4.9 Methodology	12
	4.10 Key Technological Challenges Resolved	12
	4.11 Expected Outcomes	13
5.	Bibliography / References	14

INTERNSHIP COMPLETION CERTIFICATE



Carnot Research Pvt Ltd

371 SFS Flats, Hauz Khas New Delhi - 110016

contact@carnotresearch.com

www.carnotresearch.com

GST: 07AAICC9543P1ZX

INTERNSHIP LETTER

This is to certify that Mr. Mudit Agrawal pursuing B. Tech in Computer Science from IILM University, Greater Noida, has successfully completed his internship at Carnot Research Pvt. Ltd from 1st June 2026 to 31st August 2026.

During his internship, he contributed to AI-driven projects involving government-data analysis, data-ingestion workflows, an on-premise workspace assistant with email and calendar automation, and the design and evaluation of Intel-optimised RAG workflows. He demonstrated dedication, enthusiasm, and a willingness to learn, making valuable contributions throughout his internship.

We wish him all the best in his future endeavours.

Authorized Signatory
Name: Dr Amit Oberoi
Designation: CEO
Date: 31 Aug 2026



CHAPTER 4: PROJECT DESCRIPTION

4.1 Introduction

This report gives an overview of the Summer Internship completed by Mudit Agrawal (Enrollment No. CS-2341718) a final-year B.Tech CSE (AI & ML) student from IILM University, Greater Noida as part of the curriculum for the session 2026–27. The Summer Internship took place at Carnot Research Pvt. Ltd., IITD for a duration of three months starting on 1st June 2026, Ending on 31st August 2026. Mudit Agrawal worked in the role of an AI & ML Intern.

The internship focused on researching, developing and deploying AI-powered RAG and agentic systems for the company. The tasks included building intelligent agents, designing ETL pipelines, developing email and calendar automation, implementing multiple RAG workflows and evaluating them through industry benchmarks. The experience provided real-world exposure to production-grade AI engineering, system integration, testing and on-premise deployment challenges. This report covers the projects developed, architectures and approaches used, challenges encountered and resolved, evaluation results, production integrations, and the technical knowledge gained throughout the internship.

4.2 Organization Profile

Carnot Research Pvt. Ltd. is a deep-tech AI company that builds production-grade artificial intelligence systems for government, defence and enterprise organisations. The company was founded by IIT Delhi professors and scientists who identified the need for secure AI that could operate within an organisation's own infrastructure.

Carnot Research focuses on Generative AI, Natural Language Processing and Computer Vision. Its systems are designed for mission-critical environments where reliability, security and data control are important. The company develops solutions that can operate fully on-premise, including in low-connectivity and air-gapped environments, without sending sensitive data to external cloud services.

The company develops products and solutions for areas such as enterprise knowledge management, government and defence AI, multilingual conversational systems, sports intelligence and business data analysis. Its platforms support capabilities such as multimodal document processing, Retrieval-Augmented Generation, conversational AI and intelligent workflow automation.

Carnot Research follows a research-led approach, developing solutions at the algorithm level rather than relying only on API-based systems. The company has worked with organisation's including NSG, IIT Delhi, BCG and other government and enterprise institutions, with a focus on building secure, reliable and deployable AI systems.

4.3 Problem Statement

Before this project, autonomous enterprise workflow execution had operational problems:

- **Manual document research:** Extracting critical metrics, financial figures, and structured knowledge from varied enterprise documents like PDFs, text files, internal reports, and complex spreadsheets took substantial time and human effort. This made it difficult to scale the process.
- **Manual communications:** Processing incoming inquiries, classifying sender intents, and drafting replies was done by hand with each message written and sent individually.
- **Limited thread management:** There was no system to track conversation states automatically. This meant that active proposals and pending replies were hard to keep track of and might be missed.
- **Disconnected systems:** Business operations required different interfaces such as local files, databases, email and calendars. This made it hard for the team to orchestrate workflows effectively.
- **Lack of visibility:** There was no single platform where operators could inspect active agent reasoning chains, monitor execution logs, review document pipelines, and approve high-risk automated actions before they were committed to external services.
- **Manual meeting coordination:** Scheduling discussions across calendars, checking free slots and resolving time zone conflicts required extensive manual effort.

4.4 Project Objectives

To address these operational challenges AgenticOS established key strategic project objectives:

- **Automated document intelligence:** Implement semantic retrieval and automated skill pipelines to extract key financial metrics, evaluate business risks, and synthesize structured knowledge from unstructured documents like PDFs, spreadsheets, and enterprise reports. This enables scalable and reliable decision-making processes.
- **Autonomous email communication:** Deploy natural language intent classifiers to categorize incoming messages, summarize long email threads, and draft personalized, contextual responses automatically.
- **Robust state tracking:** Build persistent database-backed thread managers to monitor multi-turn conversation lifecycles automatically. This guarantees that pending proposals, rescheduled requests, and unhandled messages are tracked without losing conversational context.
- **Unified orchestration:** Integrate diverse enterprise interfaces into an extensible harness connecting local files, databases, Gmail and calendars. This allows the system to execute end-to-end workflows smoothly.
- **Centralized operational visibility:** Provide a unified environment where operators can inspect intermediate reasoning chains, track real-time execution logs, audit document pipelines, and interactively review or approve critical automated decisions before outbound actions are dispatched.
- **Autonomous meeting coordination:** Automate calendar scheduling, detect open slots, handle timezone conversions and dispatch verified meeting invites seamlessly.

4.5 Scope of the Project

The project was implemented as a two-component platform working in conjunction with one another. The first component, the Core Agentic Harness, is responsible for document intelligence, data extraction, and skill orchestration. The second component, the Autonomous Email Pipeline, uses this agentic framework to classify message intents, manage multi-turn thread states, detect open calendar slots, coordinate meeting proposals, and execute human-in-the-loop approvals through an interactive CLI dashboard. The scope of the project covered backend development, AI agent design, database architecture, third-party API integrations, CLI interface development, and production deployment and maintenance of both platforms.

4.6 Technologies and Tools Used

The following technologies formed the core stack of AgenticOS: HARNESS-

Layer	Technology
Backend	Python 3.8+, FastAPI, Uvicorn, Setuptools, HTTPX
AI Framework	LangChain, PyTorch, Sentence-Transformers
Large Language Model	Qwen 2.5: 7B-Instruct and Llama 3.1: 8 via Ollama + OpenAI, Anthropic, Groq Cloud API (Multi-AI support)
Vector Memory and Search	ChromaDB
Tetsing and QA	pytest, pytest-cov, CI/CD, TruLens, RAGAS
Email Integration	Google API Client, OAuth 2.0, IMAPClient, BeautifulSoup4
Calendar / Meetings	Google Calendar API v3, iCalendar, pytz
Database	SQLite 3.0, SQLAlchemy, Custo DAO
Frontend	Rich(>=13.0.0), FastAPI StaticFiles, REPL Shell
Infrastructure	APScheduler, psutil, python-dotenv, Setuptools, Wheel
Doc Processing and Reporting	PyPDF2, python-docx, Pandas, OpenPyXL, ReportLab, WeasyPrint, docx2pdf, Matplotlib, Trafilatura, DuckDuckGo-Search

The Core Agentic Harness additionally used Sentence-Transformers (all-MiniLM-L6-v2) with PyTorch for local dense embeddings, ChromaDB with HNSW cosine indexing for RAG retrieval, ReportLab, WeasyPrint, and docx2pdf for automated multi-format report generation, and a declarative skill registry with dynamic chain executors for multi-step task orchestration.

4.7 System Architecture

- AgenticOS operates on a decoupled, two-tier architecture combining a Core Agentic Harness and an Autonomous Email & Scheduling Pipeline.
- The Core Harness, powered by FastAPI, ChromaDB vector storage, and a declarative skill registry, ingests enterprise documents, executes semantic RAG retrieval, extracts structured metrics, and orchestrates multi-step agentic workflows.
- Downstream, the Autonomous Email Pipeline continuously monitors incoming inboxes, performs semantic intent classification, and tracks multi-turn conversational lifecycles within an embedded SQLite database.
- Concurrently, an autonomous scheduling daemon interfaces with Google Calendar to algorithmically detect open slots, resolve time zone conflicts, and dispatch verified meeting invites.
- Bound together by an asynchronous asyncio runtime with human-in-the-loop governance guardrails, the entire system exposes its workflow execution, real-time logging, and state management through an interactive Rich-powered CLI and web dashboard.

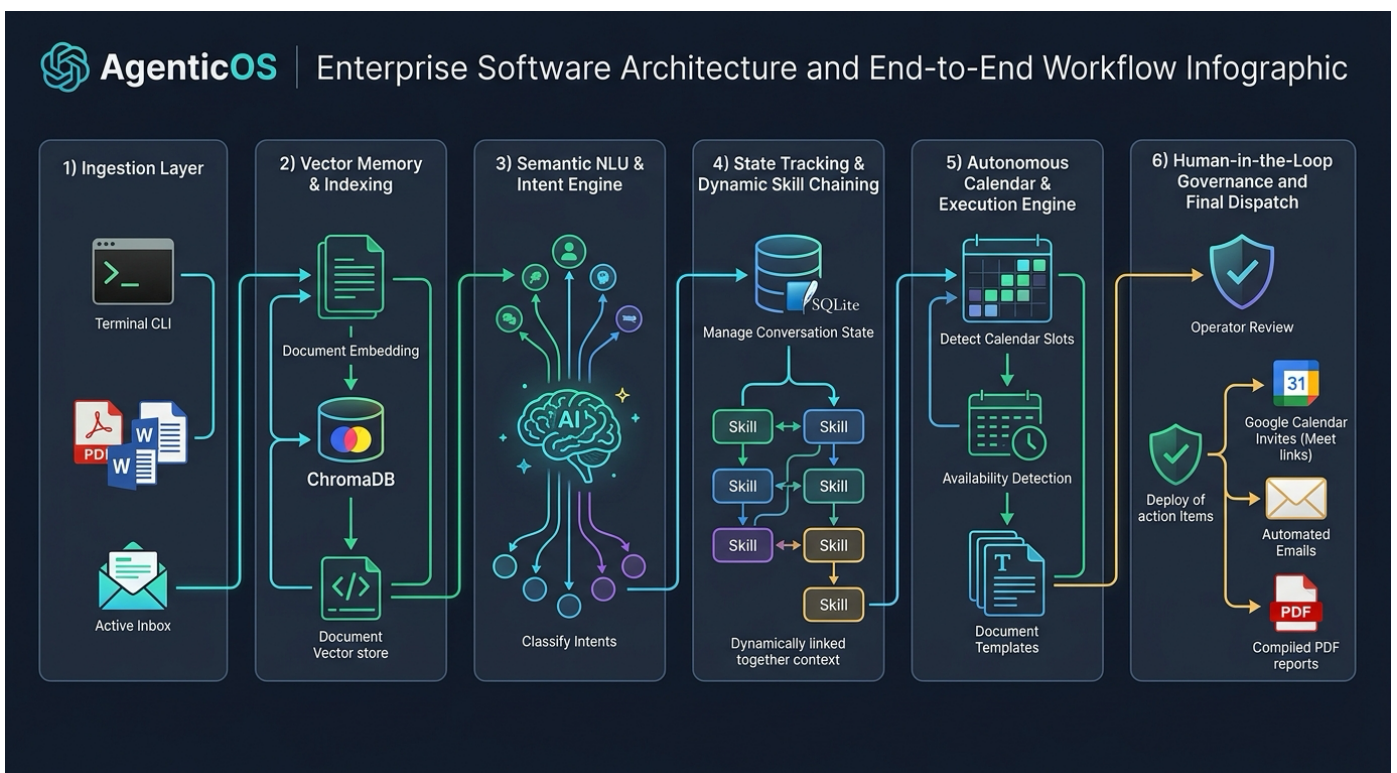
4.8 Project Flow

The AgenticOS Platform helps to analyze, automate and coordinate data across enterprise workflows and communications. The AgenticOS Autonomous Platform follows a workflow:

1. **Document Ingestion & Knowledge Indexing:** Users upload enterprise documents (PDF, DOCX, CSV, TXT) via the CLI or web interface. The system cleans and chunks the text using LangChain splitters, computes dense vector embeddings with all-MiniLM-L6-v2, and stores them in ChromaDB with HNSW cosine indexing for rapid semantic retrieval.
2. **Continuous Mailbox & Inbox Monitoring:** A background daemon (`thread_monitor.py` powered by APScheduler) continuously polls mailboxes via Google OAuth 2.0 (Gmail API) or IMAP. It extracts unread emails, parses multipart MIME payloads, sanitizes HTML bodies, and downloads attachments into the workspace sandbox.
3. **Semantic NLU & Intent Classification:** Every incoming email or interactive user query is routed through a hybrid semantic NLU engine (`intent_classifier.py`). The classifier evaluates the content against operational intents (e.g., meeting scheduling, email drafting, risk analysis, document generation) while filtering out automated system emails and bounce notifications.
4. **Multi-Turn Thread State Tracking & Deduplication:** When an email interaction begins, the

system queries the SQLite database via a custom DAO to find existing conversation records. It applies lookback deduplication (matching normalized subject and sender) to prevent duplicate threads and initializes a finite state machine (PENDING, PROPOSED, CONFIRMED, SCHEDULED, FAILED) to manage the conversation lifecycle.

5. **Dynamic Skill Chaining & Workflow Planning:** For analytical or complex tasks, the Chain Executor inspects the declarative Skill Registry (skills.json) and formulates a multi-step execution plan (e.g., `retrieve_data` → `assess_risk_level` → `generate_document` → `draft_email`). Intermediate outputs are validated and passed downstream by the Param Executor.
6. **Autonomous Slot Detection & Calendar Resolution:** If a meeting request is detected, the FreeSlotDetector queries Google Calendar API v3 to scan existing events. It computes open slots within configured working hours, adds buffer times, normalizes relative dates (e.g., "next Tuesday afternoon") using dateparser, accounts for attendee time zones, and drafts tailored meeting proposals.
7. **Human-in-the-Loop (HITL) Governance & Review:** Before executing critical external actions (such as sending emails or booking calendar events), the system invokes the Approval Workflow Engine. Operators receive interactive notifications via the Rich terminal UI, Slack, or MS Teams, allowing them to review, modify, approve, or reject the proposed action.
8. **Multi-Channel Dispatch, Artifact Generation & Audit Logging:** Once approved, the system dispatches final actions: sending outgoing emails via Gmail API/SMTP, generating RFC 5545 .ics calendar invites with Google Meet links, compiling formatted PDF/Word reports (via ReportLab and docx2pdf), and recording full audit logs and session history in SQLite.



4.9 Methodology

- **Dual-System Architecture:** AgenticOS uses a decoupled design separating the Core Agentic Harness (document RAG, skill chainer, Rich CLI) from the Autonomous Email Pipeline (background daemon, calendar scheduler), ensuring modular resilience across workflows.
- **Knowledge Processing & RAG:** Unstructured documents (PDF, DOCX, CSV) are parsed, chunked with LangChain, and converted into dense vector embeddings using SentenceTransformers (all-MiniLM-L6-v2) stored in ChromaDB for fast cosine similarity retrieval.
- **Semantic NLU & Intent Routing:** A hybrid classification pipeline applies structural regex pre-filtering to eliminate automated bot emails, followed by LLM semantic classification to route user requests into scheduling, drafting, or document intelligence intents.
- **State Tracking & Deduplication:** Multi-turn conversations are managed via an ACID-compliant SQLite finite state machine (PENDING to SCHEDULED). A 1-hour lookback window matching both normalized subject and sender eliminates duplicate proposal generation.
- **Autonomous Scheduling Engine:** The FreeSlotDetector scans Google Calendar API v3 for open windows, enforces working-hour buffers, resolves timezones via pytz, and automatically generates RFC 5545 .ics meeting invites with Google Meet links.
- **Governance & Verification:** Human-in-the-Loop guardrails stage high-risk actions for operator review and approval via CLI or Slack/Teams webhooks. The system is verified through comprehensive pytest suites, mock environments, and automated end-to-end stress tests.

4.10 Key Technical Challenges Resolved

- **Duplicate Thread Prevention** Repeated incoming emails previously triggered redundant threads and conflicting proposals. Resolved by introducing a one-hour lookback deduplication filter matching normalized subject and sender, routing subsequent replies to existing active threads.
- **State Starvation on Network Failures** Transient SMTP errors previously left threads permanently frozen in PENDING status. Resolved by adding an explicit FAILED state to the state machine, accompanied by exponential backoff and retry counters.
- **Automated Bot Filtering** Calendar notifications, newsletters, and daemon bounce messages mistakenly triggered meeting proposals. Resolved via hybrid structural regex screening that eliminates system senders before passing legitimate inquiries to the semantic NLU engine.
- **Semantic Reschedule Disambiguation** Ambiguous phrasing confused LLMs between polite remarks and genuine rescheduling requests. Resolved by introducing an explicit `is_reschedule_request` field with a 0.7 confidence threshold to prevent false-positive modifications.
- **Timezone and Participant Coordination** Disparate timezones caused slot misalignment, while CC recipients were dropped. Resolved by dynamically fetching timezone settings from Google Calendar, applying pytz localization, and parsing IMAP envelopes to track attendees relationally.

- **Concurrent Safety and Governance** Autonomous execution risked slot collisions and unauthorized messaging. Resolved by enforcing ACID SQLite transactions with unique thread constraints, coupled with Human-in-the-Loop approval queues before external dispatch.

4.11 Expected Outcomes

- **Autonomous Document Intelligence** Accelerate information extraction from unstructured enterprise documents (PDF, DOCX, CSV). The system autonomously extracts financial metrics, evaluates business risks, and compiles formatted executive reports, eliminating manual data entry and document synthesis.
- **Zero-Touch Email Triage** Eliminate email latency by deploying semantic NLU classification. Incoming messages are instantly categorized, summarized, and routed, with high-quality contextual drafts prepared automatically for routine business inquiries and follow-ups.
- **Frictionless Meeting Coordination** Automate the entire scheduling lifecycle without external links. The engine detects open calendar slots, resolves timezone differences, negotiates proposals over email, and dispatches verified Google Calendar invites containing Google Meet links.
- **Robust Conversation State Tracking** Prevent dropped leads and fragmented communication by persisting multi-turn dialogue states in SQLite. The system maintains context across ongoing negotiations and enforces deduplication to prevent duplicate proposals.
- **Unified Cross-Tool Orchestration** Break down tool silos by establishing an extensible skill harness. The platform seamlessly chains local file access, vector memory retrieval, database queries, and external APIs into cohesive, multi-step autonomous workflows.
- **Governed Enterprise Safety** Mitigate operational risks and model hallucinations through Human-in-the-Loop guardrails. Operators retain full oversight to review, edit, or approve outbound emails and calendar bookings, backed by immutable audit trails.

CHAPTER 5: BIBLIOGRAPHY / REFERENCES

1. Retrieval-Augmented Generation & Embeddings
 - Lewis et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (NeurIPS 2020, [arXiv:2005.11401](#)).
 - Reimers & Gurevych (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" (EMNLP 2019, all-MiniLM-L6-v2).
2. Agentic Reasoning & Tool Orchestration
 - Yao et al. (2022). "ReAct: Synergizing Reasoning and Acting in Language Models" (ICLR 2023, [arXiv:2210.03629](#)).
 - Schick et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools" (NeurIPS 2023, [arXiv:2302.04761](#)).
3. Vector Search & Nearest Neighbors
 - Malkov & Yashunin (2018). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs" (IEEE TPAMI, foundational HNSW algorithm in ChromaDB).
4. Messaging Protocols & Scheduling Standards
 - IETF RFC 5545 (2009). "Internet Calendaring and Scheduling Core Object Specification (iCalendar)".
 - IETF RFC 5322 (2008). "Internet Message Format" (Standard MIME specifications for email pipelines).
5. LLM Evaluation & Governance Frameworks
 - Es et al. (2023). "RAGAS: Automated Evaluation of Retrieval Augmented Generation" ([arXiv:2309.15217](#)).
 - TruEra (2023). "TruLens: Evaluation, Tracking, and Guardrails for LLM Applications" ([truera.com/trulens](#)).
6. Developer APIs & Cloud Specifications
 - Google Developers (2024). "Google Calendar API v3 and Gmail REST API Developer Guides" ([developers.google.com](#)).
 - FastAPI Documentation (2024). "FastAPI Web Framework and High-Performance Server Architecture" ([fastapi.tiangolo.com](#)).